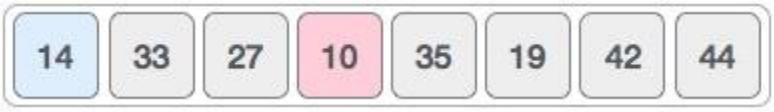
Selection Sort

Consider the following depicted array as an example.

For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.



So we replace 14 with 10. After one iteration 10, which happens to be the minimum value in the list, appears in the first position of the sorted list.

For the second position, where 33 is residing, we start scanning the rest of the list in a linear manner.

We find that 14 is the second lowest value in the list and it should appear at the second place. We swap these values.

After two iterations, two least values are positioned at the beginning in a sorted manner.

The same process is applied to the rest of the items in the array.

Following is a pictorial depiction of the entire sorting process −


10 14 27 33 35 19 42 44
10 14 27 33 35 19 42 44
10 14 19 33 35 27 42 44
10 14 19 33 35 27 42 44
10 14 19 27 35 33 42 44
10 14 19 27 35 33 42 44
10 14 19 27 35 33 42 44
10 14 19 27 33 35 42 44
10 14 19 27 33 35 42 44

Selection Sort

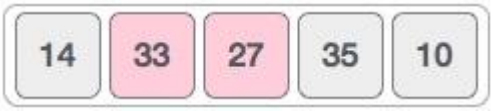
```
#include <iostream>

using namespace std;

int main()
{
   int arr[20],i,j,size,temp,loc,min;
   cout<<"enter aray size";
   cin>>size;

   cout<<"enter array elements";
   for(i=0;i<size;i++)
   {
      cin>>arr[i];
   }

   for(i=0;i<(size-1);i++)
   {
      min=arr[i];
      loc=i;

      for(j=i+1;j<size;j++)
      {
         if(arr[j]>min)
         {
            min=arr[j];
            loc=j;
         }
      }

      temp=arr[i];
      arr[i]=arr[loc];
      arr[loc]=temp;
   }

   cout<<"array after selection sort";
   for(i=0;i<size;i++)
   {
      cout<<arr[i]<<" ";
   }
}
```

Bubble Sort

Bubble sort starts with very first two elements, comparing them to check which one is greater.

In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.

We find that 27 is smaller than 33 and these two values must be swapped.



The new array should look like this −

Next we compare 33 and 35. We find that both are in already sorted positions.

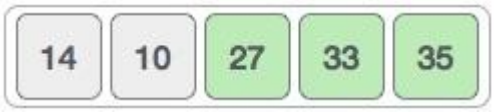
Then we move to the next two values, 35 and 10.

We know then that 10 is smaller 35. Hence they are not sorted.

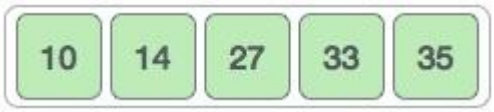
We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this −

To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this −

Notice that after each iteration, at least one value moves at the end.



And when there's no swap required, bubble sorts learns that an array is completely sorted.



Bubble Sort

```
#include <iostream>

using namespace std;

int main()
{
   int a[20],i,j,size,temp,swap;
   cout<<"enter aray size";
   cin>>size;

   cout<<"enter array element";
   for(i=0;i<size;i++)
   {
      cin>>a[i];

   }

   for(i=0;i<(size-1);i++)
   {
      swap=0;
      for(j=0;j<(size-i)-1;j++)
      {
         if(a[j] > a[j+1])
         {
```

```
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
            swap=1;
         }
      }

      if(swap==0)
      {
         break;
      }
   }

   cout<<"array after bubble sort";
   for(i=0;i<size;i++)
   {
      cout<<""<<a[i];
   }

}
```

Insertion Sort

Insertion sort compares the first two elements.

It finds that both 14 and 33 are already in ascending order. For now, 14 is in sorted sub-list.

Insertion sort moves ahead and compares 33 with 27.

And finds that 33 is not in the correct position.

It swaps 33 with 27. It also checks with all the elements of sorted sub-list. Here we see that the sorted sub-list has only one element 14, and 27 is greater than 14. Hence, the sorted sub-list remains sorted after swapping.

By now we have 14 and 27 in the sorted sub-list. Next, it compares 33 with 10.

These values are not in a sorted order.

So we swap them.

However, swapping makes 27 and 10 unsorted.

Hence, we swap them too.

Again we find 14 and 10 in an unsorted order.

We swap them again. By the end of third iteration, we have a sorted sub-list of 4 items.

Insertion Sort

```
#include <iostream>

using namespace std;

int main()
{
   int a[20],i,j,size,temp;
   cout<<"enter aray size";
   cin>>size;

   cout<<"enter array element";
   for(i=0;i<size;i++)
   {
      cin>>a[i];

   }

   for(i=1;i<size;i++)
   {
      temp=a[i];
      j=i-1;
      while(j>=0 && a[j]>temp)
      {
         a[j+1] = a[j];
         j--;
      }
      a[j+1]=temp;
   }
   cout<<"array after insertion sort";
   for(i=0;i<size;i++)
   {
      cout<<""<<a[i];
   }
}
```